

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА
з освітнього компоненту «Програмування»
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «ТРАНСПОРТНІ ЗАСОБИ»

Студентки 1 курсу, денна форма навчання
спеціальність F7 «Комп'ютерна інженерія»

Моїсєєвої Ельвіри Ігорівни

Керівник: к.ф.-м.н., доц. Антоненко О.С.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____ Антоненко О.С.
(підпис) (прізвище та ініціали)

_____ Шестопапов С.В.
(підпис) (прізвище та ініціали)

_____ _____
(підпис) (прізвище та ініціали)

ЗМІСТ

	стор.
ВСТУП.....	3
ЗАВДАННЯ	4
1.ТЕОРЕТИЧНАЧАСТИНА	5
1.1.Поняття ООП	5
1.2. Опис об'єктної частини	6
2. ПРАКТИЧНА ЧАСТИНА	7
2.1 Опис класів.....	7
2.2 Інструкція користувача.....	9
2.3 Інтерфейс	10
ВИСНОВКИ.....	13
СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ	14

ВСТУП

Метою даної курсової роботи є закріплення теоретичних знань з дисципліни та набуття практичних навичок проєктування і розробки програмного забезпечення мовою C++ з використанням об'єктно-орієнтованого підходу.

Для досягнення мети було поставлено наступні завдання:

1. Проаналізувати предметну область та виділити основні сутності для створення ієрархії класів.
2. Розробити базовий абстрактний клас та класи-спадкоємці з відповідними полями та методами.
3. Реалізувати механізми поліморфізму для виконання специфічних завдань (наприклад, розрахунку комфорту).
4. Забезпечити надійність програми за допомогою механізмів обробки виняткових ситуацій (try...catch).
5. Створити зручне консольне меню для взаємодії користувача з програмою.

У результаті виконання роботи має бути створена готова до використання інформаційна система керування автопарком, яка автоматизує процеси обліку транспортних засобів та підбору оптимального транспорту для перевезень.

ЗАВДАННЯ

Варіант 16а. Створення ієрархії класів на тему «Транспортні засоби»

Створити класи: «Транспортний засіб» (вага, максимальна швидкість, витрата палива в літрах на 100 км), «Легковий Автомобіль» (максимально можливий багаж, що перевозиться в кг, клас/сегмент – див. http://ua.wikipedia.org/wiki/Системи_класифікації_легкового_автотранспорту), тип пасажирів, тип оббивки сидінь), «Мотоцикл» (з візком або без), «Вантажний автомобіль» (вантажопідйомність), «Автобус» (кількість пасажирів, наявність кондиціонера, сидінь, що відкидаються, зручність сидінь, максимальний можливий багаж для кожного пасажирів). Реалізувати метод обчислення комфорту перевезення пасажирів у цьому транспортному засобі (для мотоцикла, легкового автомобіля, автобуса).

Реалізувати додавання нових транспортних засобів та підбір автомобілів для перевезення пасажирів та їх вантажів (принцип підбору – ручний розподіл, оптимізація комфорту та/або вартості – на вибір студента).

1. ТЕОРЕТИЧНА ЧАСТИНА

1.1. Поняття ООП

Об'єктно-орієнтоване програмування (ООП) є однією з найпоширеніших та найефективніших парадигм сучасної розробки програмного забезпечення. Її головна ідея полягає у представленні програми як сукупності об'єктів, кожен з яких є екземпляром певного класу, а класи утворюють ієрархію спадкування[1]. Використання принципів ООП, таких як інкапсуляція, успадкування та поліморфізм, дозволяє створювати гнучкий, надійний та структурований код, який легко масштабувати та модифікувати.

Кожний стиль програмування має свою концептуальну базу. Для об'єктно-орієнтованого підходу такою базою є об'єктна модель[2]. Її побудова оснований на використанні трьох основних концепцій:

- а) інкапсуляція;
- б) успадкування;
- с) поліморфізм.

Інкапсуляція – це властивість системи, що дозволяє тісно пов'язати дані та методи роботи з ними всередині класу, а також зробити дані і деталі реалізації недоступними для інших частин.

Успадкування – це відношення між класами, при якому один клас повторює структуру і поведінку іншого класу (одиначне успадкування) або інших класів (множинне успадкування).

Поліморфізм – це властивість, яка дозволяє одне і те ж ім'я використовувати для вирішення двох або більше схожих, але технічно різних задач.

1.2. Опис об'єктної частини (предметної області)

Програма реалізує систему управління автопарком, де користувач виступає в ролі диспетчера або адміністратора транспортної системи. Взаємодія з програмою відбувається через консольне меню, що дозволяє гнучко керувати базою транспортних засобів та виконувати логічні операції з підбору оптимального транспорту для конкретних завдань.

Користувач має можливість поповнювати автопарк новими одиницями техніки чотирьох категорій: легковими автомобілями, мотоциклами, вантажівками та автобусами. Під час створення нового об'єкта програма запитує специфічні характеристики для кожного типу транспорту (наприклад, наявність коляски для мотоцикла або кондиціонера для автобуса). При цьому система здійснює сувору перевірку вхідних даних за допомогою механізму виняткових ситуацій (генерація об'єктів `std::invalid_argument`)[3], автоматично блокуючи створення транспорту з від'ємною вагою, швидкістю або місткістю. Усі додані транспортні засоби зберігаються в єдиній динамічній структурі керуючого класу `TransportSystem`. Завдяки цьому користувач може у будь-який момент переглянути повний список доступного автопарку, де для кожного засобу за допомогою поліморфізму розраховується і виводиться його індивідуальний рівень комфорту.

Ключовою функцією системи є автоматизований підбір найкращого транспортного засобу для виконання конкретного рейсу за допомогою методу `findBestVehicleForJob()`. Користувач задає дві головні вимоги: необхідну кількість пасажирських місць та загальну вагу вантажу (багажу). Програма динамічно перебирає всі наявні транспортні засоби, викликає їхні перевизначені методи для перевірки місткості та вантажопідйомності, відсіюючи ті, що не підходять за габаритами. З-поміж тих засобів, які здатні виконати завдання, система автоматично обирає та пропонує користувачу той варіант, який забезпечить максимальний рівень комфорту перевезення.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Розроблена програма базується на принципах об'єктно-орієнтованого підходу. Архітектура застосунку складається з двох основних логічних блоків:

Ієрархії класів предметної області (базовий абстрактний клас `Vehicle` та його спадкоємці: `Car`, `Motorcycle`, `Truck`, `Bus`), які описують характеристики транспортних засобів.

Керуючого класу `TransportSystem`, який реалізує логіку взаємодії з об'єктами (додавання, зберігання, пошук) та приховує деталі реалізації від головної функції `main()`.

1) Клас `Vehicle` виступає абстрактним базовим класом (інтерфейсом) для всіх типів транспортних засобів. Інкапсуляція реалізована шляхом оголошення загальних характеристик у секції `private`:

- a) `weight` (вага транспортного засобу);
- b) `maxSpeed` (максимальна швидкість);
- c) `fuelConsumption` (витрата палива).

Для доступу до цих полів розроблено відповідні публічні методи-гетери. У конструкторії базового класу передбачена базова валідація вхідних даних: якщо вага, швидкість або витрата палива менші або дорівнюють нулю, генерується виняткова ситуація `std::invalid_argument`.

Для забезпечення поліморфізму в класі оголошено чисті віртуальні методи (pure virtual functions)[4]:

- a) `virtual void displayInfo() const = 0;` - для виведення інформації про транспорт;
- b) `virtual int calculateComfort() const = 0;` - для розрахунку рівня комфорту;
- c) `virtual int getPassengerCapacity() const = 0;`
- d) `virtual double getCargoCapacity() const = 0;` - для отримання пасажиромісткості та вантажопідйомності відповідно.

Також у базовому класі визначено віртуальний деструктор `virtual ~Vehicle()`, що є обов'язковою умовою для коректного вивільнення динамічної пам'яті об'єктів-спадкоємців.

Кожен із класів-спадкоємців розширює базовий клас `Vehicle`, додаючи специфічні характеристики та перевизначаючи віртуальні методи (з використанням специфікатора `override`):

2) Клас `Car` (Легковий автомобіль): Доповнений параметрами максимального багажу, сегменту автомобіля (`enum CarSegment`), типу кузова (`enum BodyType`), кількості пасажирів та типу оббивки салону (`enum SeatUpholsteryType`). Метод `calculateComfort()` реалізовано з урахуванням базових балів, до яких додаються бонусні бали за покращену оббивку (алькантара, натуральна шкіра) та тип кузова (кабріолет);

3) Клас `Motorcycle` (Мотоцикл): Містить булеве поле `isHasASidecar`, яке визначає наявність коляски. Від цього параметра залежить як місткість (1 або 2 пасажирів), так і рівень комфорту перевезення;

4) Клас `Truck` (Вантажівка): Орієнтований виключно на перевезення вантажів, тому доповнений полем `liftingCapacity`. Оскільки вантажівки не призначені для пасажирів, методи `calculateComfort()` та `getPassengerCapacity()` повертають нуль;

5) Клас `Bus` (Автобус): Описує транспорт для масових перевезень. Містить поля кількості пасажирів, наявності кондиціонера, відкидних сидінь, рівня комфорту сидінь та доступного багажу на одну особу. Рівень комфорту розраховується сумуванням базового комфорту сидінь та додаткових балів за наявність кондиціонера і відкидних спинок.

6) Для управління масивом транспортних засобів розроблено клас `TransportSystem`. Для збереження даних використовується динамічний масив вказівників на базовий клас (`Vehicle vehicles`), що дозволяє зберігати об'єкти будь-яких класів-спадкоємців в одній структурі даних (реалізація поліморфізму підтипів)[5].

a) `addVehicle(Vehicle* vehicle)` - додає новий об'єкт до масиву. Якщо масив заповнений, автоматично викликається приватний метод `resizeArray()`, який збільшує місткість масиву (`maxCapacity`) вдвічі та перерозподіляє пам'ять, що запобігає переповненню;

b) `displayAllVehicles()` - ітерується по масиву та поліморфно викликає метод `displayInfo()` для кожного об'єкта;

c) `findBestVehicleForJob(int requiredPassengers, double totalCargoWeight)` - реалізує алгоритм пошуку оптимального транспортного засобу за критерієм максимального комфорту. Метод перевіряє, чи відповідає транспорт вимогам щодо кількості пасажирів та ваги вантажу, після чого порівнює їхній рівень комфорту і виводить інформацію про найкращий знайдений варіант.

2.2 Інструкція користувача

Розроблений програмний застосунок реалізований у вигляді консольного додатка. Після запуску файлу програми користувач бачить на екрані головне меню, яке містить перелік доступних дій для керування автопарком. Управління програмою здійснюється в інтерактивному режимі: користувач повинен ввести з клавіатури номер відповідного пункту меню та натиснути клавішу «Enter».

Головне меню пропонує наступні опції:

1-4 - Додавання нового транспортного засобу (Легковий автомобіль, Мотоцикл, Вантажівка або Автобус відповідно). При виборі одного з цих пунктів програма послідовно запитує у користувача необхідні характеристики (вагу, швидкість, кількість пасажирів тощо).

5 - Виведення списку всіх зареєстрованих в системі транспортних засобів із зазначенням їхнього розрахованого рівня комфорту.

6 - Підбір найкращого транспортного засобу. Програма запитує необхідну кількість пасажирів та загальну вагу багажу, після чого здійснює пошук в автопарку та виводить дані про транспортний засіб, який відповідає цим вимогам і має найвищий рівень комфорту.

0 - Завершення роботи програми та вихід із системи.

2.3 Інтерфейс

Для перевірки коректності роботи логіки програми та алгоритмів було проведено серію тестів.

Тест 1. Перевірка роботи головного меню та додавання об'єктів. Користувач успішно запускає програму, обирає пункти меню для додавання різних типів транспорту та вводить валідні (коректні) дані з клавіатури. Програма підтверджує успішне збереження об'єктів у масив.

```

--- FLEET MENU ---
1. Add Car
2. Add Motorcycle
3. Add Truck
4. Add Bus
5. Show all vehicles
6. Find best vehicle for the trip
0. Exit
Your choice: 1

--- Enter parameters for Car ---
Weight (kg): 1400
Max Speed (km/h): 320
Fuel Consumption (L/100km): 13
Max Baggage (kg): 50
Passenger Count: 2
Car Segment (0-A, 1-B, 2-C, 3-D, 4-E, 5-F, 6-J, 7-S, 8-M): 1
Body Type (0-Sedan, 1-Wagon, 2-Cabriolet): 1
Seat Upholstery (0-Cloth, 1-Suede, 2-Vinyl, 3-Alcantara, 4-Eco_Leather, 5-Genuine_Leather): 5
Car added successfully!

```

Рисунок 2.1 – Головне меню

Тест 2. Виведення інформації про стан автопарку. Після додавання кількох одиниць транспорту здійснюється виклик пункту меню «5». Програма коректно

ітерується по масиву об'єктів, застосовуючи поліморфізм, і виводить форматований список автомобілів з їхніми унікальними полями та розрахованими балами комфорту.

```

--- FLEET MENU ---
1. Add Car
2. Add Motorcycle
3. Add Truck
4. Add Bus
5. Show all vehicles
6. Find best vehicle for the trip
0. Exit
Your choice: 5
1. [Car]          Weight: 1400 kg
                  Max Speed: 320 km/h
                  Fuel Consumption: 13 L/100km
                  Max Baggage: 50 kg
                  Car Segment: B
                  Body Type: Wagon
                  Passenger Count: 2
                  Seat Upholstery Type: Genuine leather
                  Comfort Level: 7
2. [Truck]        Weight: 300 kg
                  Max Speed: 120 km/h
                  Fuel Consumption: 3 L/100km
                  Lifting capacity: 23000 kg
                  Comfort Level: 0

```

Рисунок 2.2 – Виведення інформації про стан автопарку

Тест 3. Підбір оптимального транспорту. Здійснюється перевірка алгоритму пошуку (пункт меню «6»). Вводяться тестові вимоги: наприклад, необхідність перевезти 4 пасажирів. Програма здійснює перебір наявних об'єктів, відкидає ті, що не підходять за місткістю (наприклад, мотоцикли чи вантажівки), знаходить серед решти варіант із найвищим параметром комфорту та виводить результат.

```

--- Find The Best Vehicle ---
How many passengers need to be transported? 2
What is the total weight of the cargo (kg)? 20

Best vehicle is:
[Car]          Weight: 1400 kg
                  Max Speed: 320 km/h
                  Fuel Consumption: 13 L/100km
                  Max Baggage: 50 kg
                  Car Segment: B
                  Body Type: Wagon
                  Passenger Count: 2
                  Seat Upholstery Type: Genuine leather

```

Рисунок 2.3 – Підбір оптимального транспорту

Тест 4. Обробка виняткових ситуацій (захист від некоректних даних). Перевіряється надійність системи. Користувач навмисно вводить некоректні дані: від'ємне значення ваги або місткості при створенні транспортного засобу. Блок перевірки у конструкторі генерує виняток, який успішно перехоплюється блоком try...catch у головній функції. Замість аварійного завершення роботи програма виводить повідомлення про помилку і дозволяє продовжити роботу.

```
--- Enter parameters for Car ---  
Weight (kg): 0  
Max Speed (km/h): 10  
Fuel Consumption (L/100km): 4  
Max Baggage (kg): 1  
Passenger Count: 1  
Car Segment (0-A, 1-B, 2-C, 3-D, 4-E, 5-F, 6-J, 7-S, 8-M): 0  
Body Type (0-Sedan, 1-Wagon, 2-Cabriolet): 2  
Seat Upholstery (0-Cloth, 1-Suede, 2-Vinyl, 3-Alcantara, 4-Eco_Leather, 5-Genuine_Leather): 0  
  
ERROR: ERROR! Weight, max speed and fuel consumption must be more than 0.  
Try again.
```

Рисунок 2.4 – Обробка виняткових ситуацій

ВИСНОВКИ

У ході виконання курсової роботи було успішно спроектовано та реалізовано програмний застосунок мовою C++ для роботи з ієрархією класів «Транспортні засоби».

Під час розробки були практично застосовані всі ключові принципи об'єктно-орієнтованого програмування. Завдяки інкапсуляції забезпечено безпечний доступ до даних об'єктів через гетери та спеціальні методи. Використання механізму успадкування дозволило уникнути дублювання коду, винісши загальні характеристики (вагу, швидкість, витрату палива) у базовий абстрактний клас `Vehicle`. Поліморфізм був реалізований за допомогою віртуальних методів (зокрема, `calculateComfort()` та `displayInfo()`), що дозволило керуючому класу `TransportSystem` працювати з масивом вказівників на базовий клас, викликаючи специфічні методи для кожного конкретного типу транспорту.

Програма успішно вирішує поставлені завдання: дозволяє додавати нові транспортні засоби до автопарку, виводити інформацію про них та здійснювати автоматизований підбір найкращого транспорту за критерієм максимального комфорту для заданої кількості пасажирів та ваги багажу.

Також у роботі реалізовано механізми захисту від некоректного введення даних користувачем за допомогою обробки винятків (`std::invalid_argument` та блоки `try...catch`), що робить програму надійною та стійкою до помилок. Набуті навички стануть міцним підґрунтям для подальшого вивчення складніших концепцій програмування та розробки великих програмних систем.

СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

1. Страуструп Б. Мова програмування C++. Спеціальне видання / Б. Страуструп. – Київ : Діасофт, 2001. – 1056 с.
2. Прата С. Мова програмування C++. Лекції та вправи. 6-те видання / С. Прата. – Київ : Діалектика, 2012. – 1248 с.
3. Документація мови C++ (C++ Reference) [Електронний ресурс]. – Режим доступу: <https://en.cppreference.com/>
4. Шилдт Г. C++. Базовий курс. 3-тє видання / Г. Шилдт. – Київ : Вільямс, 2010. – 624 с.
5. Васильєв О. М. Програмування на C++ в прикладах і задачах / О. М. Васильєв. – Київ : Ліра-К, 2017. – 382 с.
6. Репозиторій GitHub на курсовий проєкт - https://github.com/elviramoyiseeva0/coursework_MOISIEIEVA

Додаток А. Діаграма класів

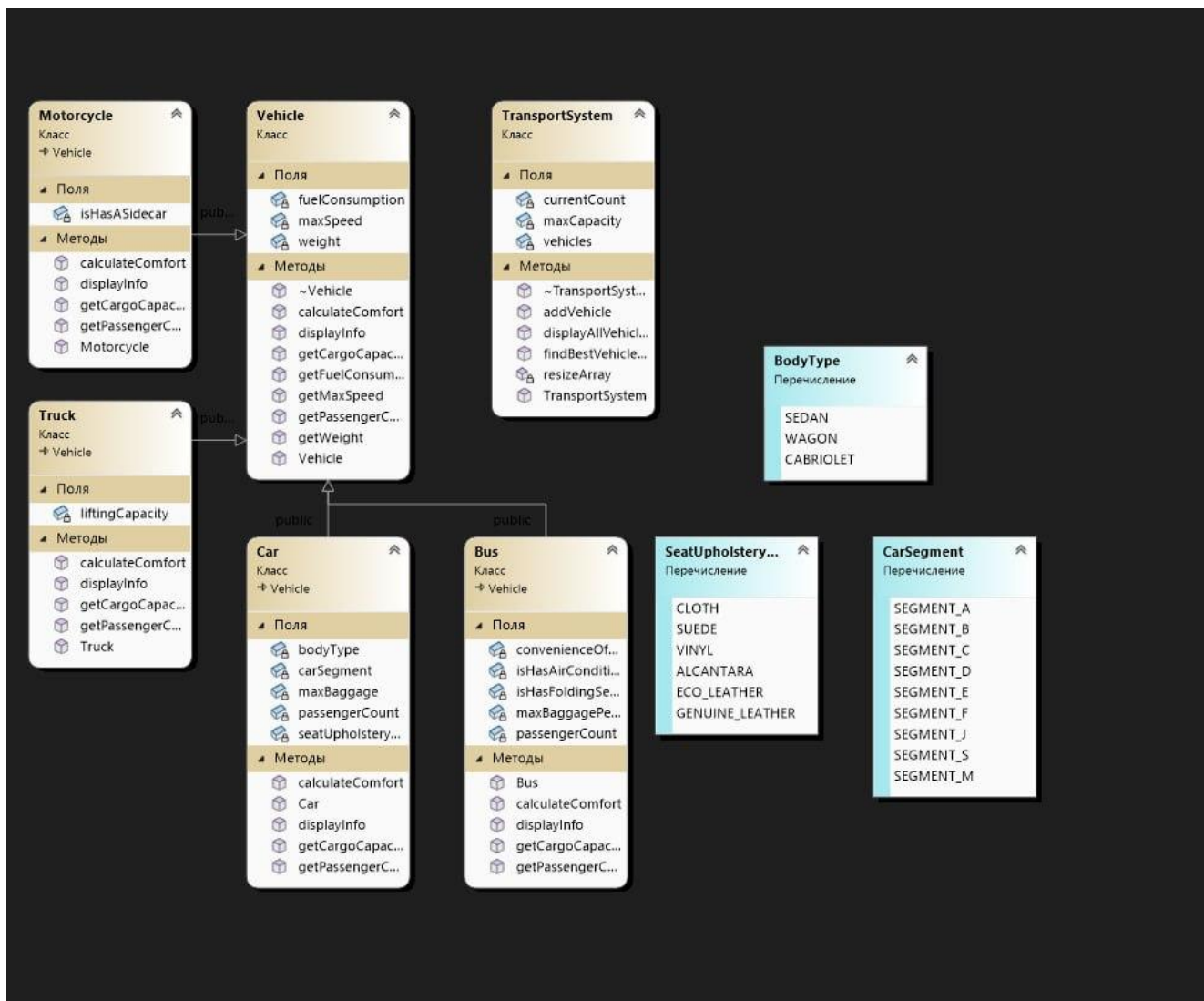


Рисунок А.1 – Діаграма класів

Додаток Б. Код класу

```
#pragma once
#include <string>

class Vehicle
{
private:
    double weight;
    double maxSpeed;
    double fuelConsumption;

public:
    Vehicle(double weight_, double maxSpeed_, double fuelConsumption_);
    virtual ~Vehicle();

    double getWeight() const;
    double getMaxSpeed() const;
    double getFuelConsumption() const;

    virtual void displayInfo() const = 0;
    virtual int calculateComfort() const = 0;

    virtual int getPassengerCapacity() const = 0;
    virtual double getCargoCapacity() const = 0;
};

#include "Vehicle.h"
#include <stdexcept>

Vehicle::Vehicle(double weight_, double maxSpeed_, double fuelConsumption_)
{
    if (weight_ <= 0 || maxSpeed_ <= 0 || fuelConsumption_ <= 0)
    {
        throw std::invalid_argument("ERROR! Weight, max speed and fuel consumption must
be more than 0.");
    }
    weight = weight_;
    maxSpeed = maxSpeed_;
    fuelConsumption = fuelConsumption_;
}

Vehicle::~~Vehicle() {}

double Vehicle::getWeight() const { return weight; }
double Vehicle::getMaxSpeed() const { return maxSpeed; }
double Vehicle::getFuelConsumption() const { return fuelConsumption; }
```

Лістинг Б.1 – Клас Vehicle

```
#pragma once
#include "Vehicle.h"

enum CarSegment { SEGMENT_A, SEGMENT_B, SEGMENT_C, SEGMENT_D, SEGMENT_E, SEGMENT_F,
SEGMENT_J, SEGMENT_S, SEGMENT_M };
enum BodyType { SEDAN, WAGON, CABRIOLET };
enum SeatUpholsteryType { CLOTH, SUEDE, VINYL, ALCANTARA, ECO_LEATHER, GENUINE_LEATHER };

class Car : public Vehicle
{
private:
    double maxBaggage;
    CarSegment carSegment;
    BodyType bodyType;
```

Лістинг Б.2 – Клас Car, аркуш 1

```

    int passengerCount;
    SeatUpholsteryType seatUpholsteryType;

public:
    Car(double weight_, double maxSpeed_, double fuelConsumption_, double maxBaggage_,
        CarSegment carSegment_, BodyType bodyType_, int passengerCount_, SeatUpholsteryType
        seatUpholsteryType_);

    void displayInfo() const override;
    int calculateComfort() const override;
    int getPassengerCapacity() const override;
    double getCargoCapacity() const override;
};

#include "Car.h"
#include <iostream>
#include <stdexcept>

Car::Car(double weight_, double maxSpeed_, double fuelConsumption_, double maxBaggage_,
    CarSegment carSegment_, BodyType bodyType_, int passengerCount_, SeatUpholsteryType
    seatUpholsteryType_)
    : Vehicle(weight_, maxSpeed_, fuelConsumption_)
{
    if (maxBaggage_ <= 0 || passengerCount_ <= 0)
    {
        throw std::invalid_argument("ERROR! Max baggage and number of passengers must be
more than 0. Try again.");
    }
    maxBaggage = maxBaggage_;
    carSegment = carSegment_;
    bodyType = bodyType_;
    passengerCount = passengerCount_;
    seatUpholsteryType = seatUpholsteryType_;
}

void Car::displayInfo() const
{
    std::cout << "[Car] Number of passengers: " << passengerCount << ", Max baggage: " <<
maxBaggage << " kg\n";
}

int Car::calculateComfort() const
{
    int comfort = 5;
    if (seatUpholsteryType == ALCANTARA || seatUpholsteryType == GENUINE_LEATHER)
    {
        comfort += 2;
    }
    if (bodyType == CABRIOLET)
    {
        comfort += 1;
    }
    return comfort;
}

int Car::getPassengerCapacity() const { return passengerCount; }
double Car::getCargoCapacity() const { return maxBaggage; }

```

Лістинг Б.2 – Клас Car, аркуш 2

```

#pragma once
#include "Vehicle.h"

class Bus : public Vehicle
{

```

Лістинг Б.3 – Клас Bus, аркуш 1

```

private:
    int passengerCount;
    bool isHasAirConditioning;
    bool isHasFoldingSeats;
    int convenienceOfVisions;
    double maxBaggagePerPassenger;

public:
    Bus(double weight_, double maxSpeed_, double fuelConsumption_, int passengerCount_,
        bool isHasAirConditioning_, bool isHasFoldingSeats_, int convenienceOfVisions_, double
        maxBaggagePerPassenger_);

    void displayInfo() const override;
    int calculateComfort() const override;
    int getPassengerCapacity() const override;
    double getCargoCapacity() const override;
};

#include "Bus.h"
#include <iostream>
#include <stdexcept>
Bus::Bus(double weight_, double maxSpeed_, double fuelConsumption_, int passengerCount_,
    bool isHasAirConditioning_, bool isHasFoldingSeats_, int convenienceOfVisions_, double
    maxBaggagePerPassenger_)
    : Vehicle(weight_, maxSpeed_, fuelConsumption_) {
    if (passengerCount_ <= 0 || convenienceOfVisions_ < 1 || convenienceOfVisions_ > 10
    || maxBaggagePerPassenger_ < 0)
    {
        throw std::invalid_argument("ERROR! Incorrect data for bus. Try again.");
    }
    passengerCount = passengerCount_;
    isHasAirConditioning = isHasAirConditioning_;
    isHasFoldingSeats = isHasFoldingSeats_;
    convenienceOfVisions = convenienceOfVisions_;
    maxBaggagePerPassenger = maxBaggagePerPassenger_;
}

void Bus::displayInfo() const
{
    std::cout << "[Bus] Number of passangers: " << passengerCount << ", Air conditioning:
    " << (isHasAirConditioning ? "Yes" : "No") << "\n";
}

int Bus::calculateComfort() const
{
    int comfort = convenienceOfVisions;
    if (isHasAirConditioning)
    {
        comfort += 2;
    }
    if (isHasFoldingSeats)
    {
        comfort += 2;
    }
    return comfort;
}

int Bus::getPassengerCapacity() const { return passengerCount; }
double Bus::getCargoCapacity() const { return passengerCount * maxBaggagePerPassenger; }

```

Лістинг Б.3 – Клас Bus, аркуш 2

```
#pragma once
#include "Vehicle.h"

class Motorcycle : public Vehicle
{
private:
    bool isHasASidecar;

public:
    Motorcycle(double weight_, double maxSpeed_, double fuelConsumption_, bool
isHasASidecar_);

    void displayInfo() const override;
    int calculateComfort() const override;
    int getPassengerCapacity() const override;
    double getCargoCapacity() const override;
};
```

Лістинг Б.3 – Клас Bus, аркуш 3

```
#include "Motorcycle.h"
#include <iostream>

Motorcycle::Motorcycle(double weight_, double maxSpeed_, double fuelConsumption_, bool
isHasASidecar_)
    : Vehicle(weight_, maxSpeed_, fuelConsumption_), isHasASidecar(isHasASidecar_)
{}

void Motorcycle::displayInfo() const
{
    std::cout << "[Motorcycle] Weight: " << getWeight() << " kg, Max speed: " <<
getMaxSpeed()
        << " km/h, Sidecar: " << (isHasASidecar ? "Yes" : "No") << "\n";
}

int Motorcycle::calculateComfort() const
{
    if (isHasASidecar)
    {
        return 4;
    }
    return 2;
}

int Motorcycle::getPassengerCapacity() const
{
    if (isHasASidecar)
    {
        return 2;
    }
    return 1;
}

double Motorcycle::getCargoCapacity() const
{
    return 0;
}

#pragma once
#include "Vehicle.h"

class Truck : public Vehicle
```

Лістинг Б.4 – Клас Motorcycle, аркуш 1

```

{
private:
    double liftingCapacity;

public:
    Truck(double weight_, double maxSpeed_, double fuelConsumption_, double
liftingCapacity_);

    void displayInfo() const override;
    int calculateComfort() const override;
    int getPassengerCapacity() const override;
    double getCargoCapacity() const override;
};

```

Лістинг Б.4 – Клас Motorcycle, аркуш 2

```

#include "Truck.h"
#include <iostream>
#include <stdexcept>

Truck::Truck(double weight_, double maxSpeed_, double fuelConsumption_, double
liftingCapacity_)
    : Vehicle(weight_, maxSpeed_, fuelConsumption_)
{
    if (liftingCapacity_ <= 0) {
        throw std::invalid_argument("ERROR! Lifting capacity must be more than 0.");
    }
    liftingCapacity = liftingCapacity_;
}

void Truck::displayInfo() const
{
    std::cout << "[Truck] Lifting capacity: " << liftingCapacity << " kg\n";
}

int Truck::calculateComfort() const
{
    return 0;
}

int Truck::getPassengerCapacity() const { return 0; }
double Truck::getCargoCapacity() const { return liftingCapacity; }

#pragma once
#include "Vehicle.h"

class TransportSystem
{
private:
    Vehicle** vehicles;
    int currentCount;
    int maxCapacity;

    void resizeArray();

public:
    TransportSystem();
    ~TransportSystem();

    void addVehicle(Vehicle* vehicle);
    void displayAllVehicles() const;
    void findBestVehicleForJob(int requiredPassengers, double totalCargoWeight) const;
};

```

Лістинг Б.5 – Клас Truck

```

#include "TransportSystem.h"
#include <iostream>

TransportSystem::TransportSystem()
{
    maxCapacity = 5;
    currentCount = 0;
    vehicles = new Vehicle * [maxCapacity];
}

TransportSystem::~~TransportSystem()
{
    for (int i = 0; i < currentCount; ++i)
    {
        delete vehicles[i];
    }
    delete[] vehicles;
}

void TransportSystem::resizeArray()
{
    maxCapacity *= 2;
    Vehicle** newVehicles = new Vehicle * [maxCapacity];
    for (int i = 0; i < currentCount; ++i)
    {
        newVehicles[i] = vehicles[i];
    }
    delete[] vehicles;
    vehicles = newVehicles;
}

void TransportSystem::addVehicle(Vehicle* vehicle)
{
    if (currentCount == maxCapacity)
    {
        resizeArray();
    }
    vehicles[currentCount] = vehicle;
    currentCount++;
}

void TransportSystem::displayAllVehicles() const
{
    if (currentCount == 0)
    {
        std::cout << "The vehicle fleet is empty.\n";
        return;
    }
    for (int i = 0; i < currentCount; ++i)
    {
        std::cout << i + 1 << ". ";
        vehicles[i]->displayInfo();
        std::cout << "    Comfort Level: " << vehicles[i]->calculateComfort() << "\n";
    }
}

void TransportSystem::findBestVehicleForJob(int requiredPassengers, double
totalCargoWeight) const

```

Лістинг Б.6 – Клас TransportSystem, аркуш 1

```

{
    int theBestVehicle = -1;
    int maxComfortLevel = -1;

    for (int i = 0; i < currentCount; ++i)
    {
        if (vehicles[i]->getPassengerCapacity() >= requiredPassengers &&
            vehicles[i]->getCargoCapacity() >= totalCargoWeight)
        {

            int currentComfort = vehicles[i]->calculateComfort();
            if (currentComfort > maxComfortLevel)
            {
                maxComfortLevel = currentComfort;
                theBestVehicle = i;
            }
        }
    }

    if (theBestVehicle != -1)
    {
        std::cout << "\nBest vehicle is: \n";
        vehicles[theBestVehicle]->displayInfo();
    }
    else
    {
        std::cout << "\nUnfortunately, no suitable vehicle was found for these
requirements.\n";
    }
}

```

Лістинг Б.6 – Клас TransportSystem, аркуш 2